

Provisioning an NGINX Server with Terraform on Windows

Contents

Introduction.....	3
Prerequisites.....	3
Step 1: Start Docker Desktop.....	4
Step 2: Set Up the Project Directory	4
Step 3: Create the Terraform Configuration File.....	5
Step 4: Initialize the Project.....	7
Step 5: Apply the Configuration.....	7
Step 6: Verify the NGINX Container	9
Step 7: Destroy the Container	9
Conclusion	11

Table of Figures

Figure 1: Create and open a file.....	5
Figure 2: Terraform Configuration File.....	6
Figure 3: Initialize the Project.....	7
Figure 4: Pull the NGINX image and start container	8
Figure 5: Pull the NGINX image and start the container.....	8
Figure 6: Verify the NGINX server	9
Figure 7: Display the created image with Docker	9
Figure 8: Stop and remove the container	10
Figure 9: Stop and remove the container	10
Figure 10: Check with Docker	11

Introduction

In today's DevOps-driven world, automating infrastructure setup has become a vital skill. This tutorial is a hands-on guide I created to demonstrate how to provision an NGINX web server using Terraform and Docker on a Windows machine. My goal is to show how quickly and easily you can get a containerized application up and running with just a few lines of code.

Whether you're new to Infrastructure as Code (IaC) or looking for a practical example using popular tools like Docker and Terraform, this guide walks you through each step from setting up your environment to launching and tearing down a web server, using straightforward commands and explanations. By the end of this tutorial, you'll not only have a better understanding of how Terraform works with Docker, but you'll also have a running NGINX container you can access right from your browser.

Prerequisites

Before starting, I had ensured the following were installed on my Windows machine:

- Windows Command Prompt
- Docker Desktop for Windows: <https://www.docker.com/products/docker-desktop/>
- Terraform: <https://developer.hashicorp.com/terraform/downloads>

Step 1: Start Docker Desktop

Once Docker Desktop is installed:

1. Open the Start Menu.
2. Search for Docker Desktop and launch it.
3. Wait for the Docker whale icon in the system tray to stabilize. This indicates Docker is ready.

➤ Docker must be running to proceed with this tutorial

Step 2: Set Up the Project Directory

- Open Windows Command Prompt and run the following commands:

```
mkdir learn-terraform-docker-container
```

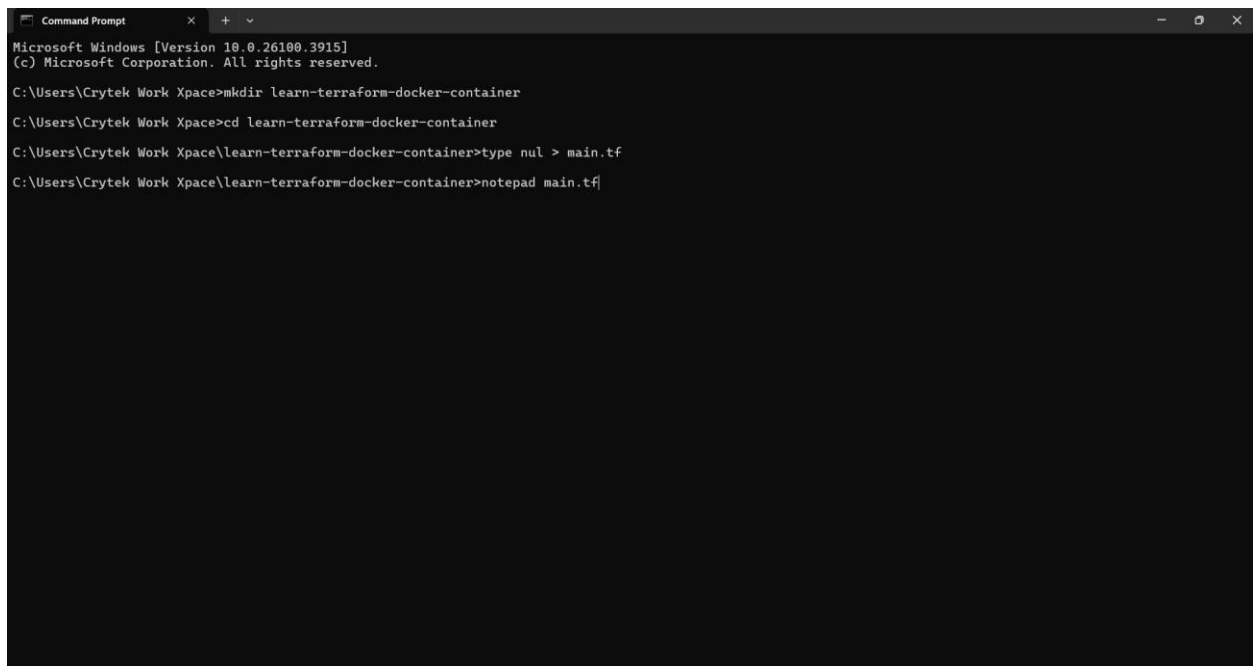
```
cd learn-terraform-docker-container
```

- Next, create an empty file named main.tf:

```
type nul > main.tf
```

- Then open the file using Notepad:

```
notepad main.tf
```



```
Microsoft Windows [Version 10.0.26100.3915]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Crytek Work Xpace>mkdir learn-terraform-docker-container
C:\Users\Crytek Work Xpace>cd learn-terraform-docker-container
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>type nul > main.tf
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>notepad main.tf
```

Figure 1: Create and open a file

Step 3: Create the Terraform Configuration File

- Paste the following code into main.tf:

```
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 3.0.1"
    }
  }
}

provider "docker" {
```

```

    host = "npipe://.//pipe//docker_engine"
}

resource "docker_image" "nginx" {

    name      = "nginx"

    keep_locally = false
}

resource "docker_container" "nginx" {

    image = docker_image.nginx.image_id

    name = "tutorial"

    ports {

        internal = 80

        external = 8000

    }

}

```

The screenshot shows a Windows Command Prompt window in the background with the following text:

```

Microsoft Windows [Version 10.0.26100.3915]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Crytek Work Xpac...
C:\Users\Crytek Work Xpac...
C:\Users\Crytek Work Xpac...
C:\Users\Crytek Work Xpac...

```

In the foreground, a web browser window displays a Terraform configuration file. The code in the browser matches the code shown in the previous block:

```

terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 3.0.1"
    }
  }
}

provider "docker" {
  host = "npipe://.//pipe//docker_engine"
}

resource "docker_image" "nginx" {
  name      = "nginx"
  keep_locally = false
}

resource "docker_container" "nginx" {
  image = docker_image.nginx.image_id
  name = "tutorial"

  ports {
    internal = 80
    external = 8000
  }
}

```

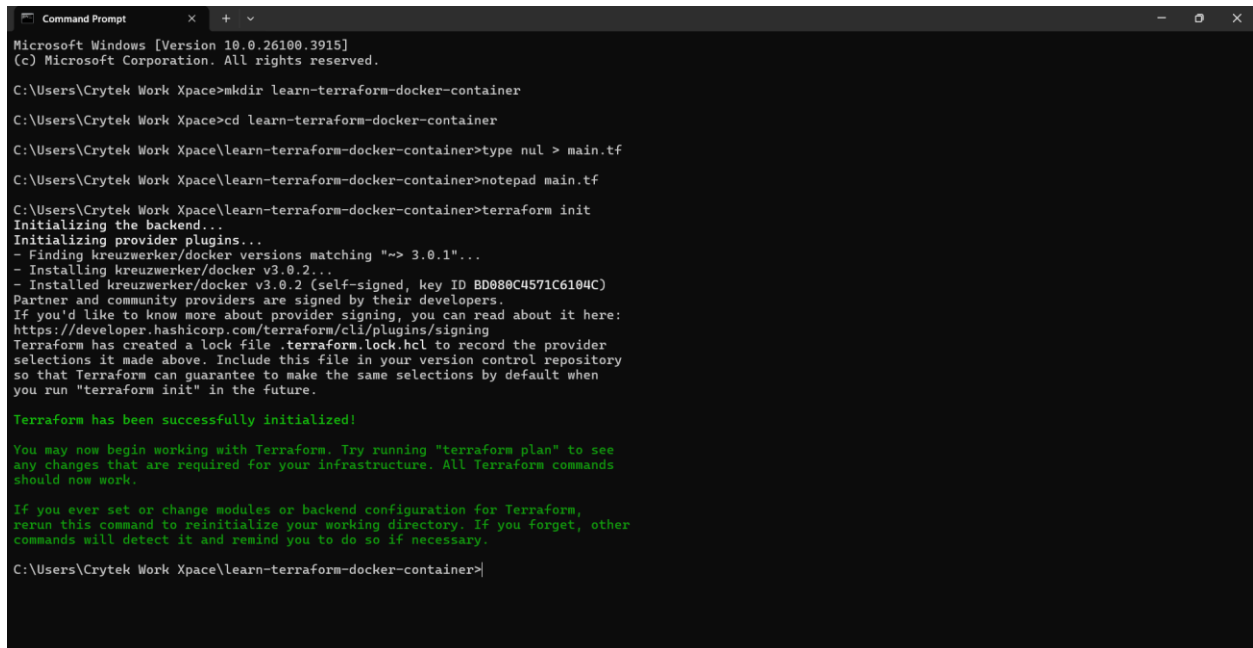
The browser window has a dark theme and shows the file path as `C:\Users\Crytek Work Xpac...`. The status bar at the bottom of the browser indicates "Ln 28, Col 1", "434 characters", "100%", "Windows (CRLF)", and "UTF-8".

Figure 2: Terraform Configuration File

Step 4: Initialize the Project

- Initialize the Terraform project to download the necessary Docker provider plugin:

terraform init



```
Microsoft Windows [Version 10.0.26100.3915]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Crytek Work Xpace>mkdir learn-terraform-docker-container
C:\Users\Crytek Work Xpace>cd learn-terraform-docker-container
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>type nul > main.tf
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>notepad main.tf
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0.1"...
- Installing kreuzwerker/docker v3.0.2...
- Installed kreuzwerker/docker v3.0.2 (self-signed, key ID BD080C4571C6104C)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

C:\Users\Crytek Work Xpace\learn-terraform-docker-container>
```

Figure 3: Initialize the Project

Step 5: Apply the Configuration

- Provision the NGINX server container:

terraform apply

```
Command Prompt - terraform x + v
commands will detect it and remind you to do so if necessary.
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# docker_container.nginx will be created
+ resource "docker_container" "nginx" {
  + attach          = false
  + bridge          = (known after apply)
  + command         = (known after apply)
  + container_logs  = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + entrypoint      = (known after apply)
  + env            = (known after apply)
  + exit_code       = (known after apply)
  + hostname       = (known after apply)
  + id             = (known after apply)
  + image          = (known after apply)
  + init           = (known after apply)
  + ipc_mode       = (known after apply)
  + log_driver     = (known after apply)
  + logs           = false
  + must_run       = true
  + name           = "tutorial"
  + network_data   = (known after apply)
  + read_only      = false
  + remove_volumes = true
  + restart        = "no"
  + rm            = false
  + runtime        = (known after apply)
  + security_opts  = (known after apply)
  + shm_size       = (known after apply)
  + start         = true
  + stdin_open     = false
  + stop_signal    = (known after apply)
  + stop_timeout   = (known after apply)
  + tty           = false
}
```

Figure 4: Pull the NGINX image and start the container

- When prompted, type ‘yes’ and press ENTER.

```
Command Prompt x + v
+ internal = 80
+ ip       = "0.0.0.0"
+ protocol = "tcp"
}

# docker_image.nginx will be created
+ resource "docker_image" "nginx" {
  + id           = (known after apply)
  + image_id     = (known after apply)
  + keep_locally = false
  + name        = "nginx"
  + repo_digest = (known after apply)
}

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

docker_image.nginx: Creating...
docker_image.nginx: Still creating... [10s elapsed]
docker_image.nginx: Still creating... [20s elapsed]
docker_image.nginx: Still creating... [30s elapsed]
docker_image.nginx: Still creating... [40s elapsed]
docker_image.nginx: Still creating... [50s elapsed]
docker_image.nginx: Still creating... [1m0s elapsed]
docker_image.nginx: Still creating... [1m10s elapsed]
docker_image.nginx: Still creating... [1m20s elapsed]
docker_image.nginx: Still creating... [1m30s elapsed]
docker_image.nginx: Still creating... [1m40s elapsed]
docker_image.nginx: Creation complete after 1m44s [id=sha256:a838707172e8069c09cf6c67a04e23e5a1a332d70a90a54999b76273a928b9cenginx]
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 1s [id=a20cd0e76442288b6ebb01acc01fdbcb874e42d02ad21873095304d2453618a8c]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>
```

Figure 5: Pull the NGINX image and start the container

Step 6: Verify the NGINX Container

- Open a web browser and go to:

start <http://localhost:8000>

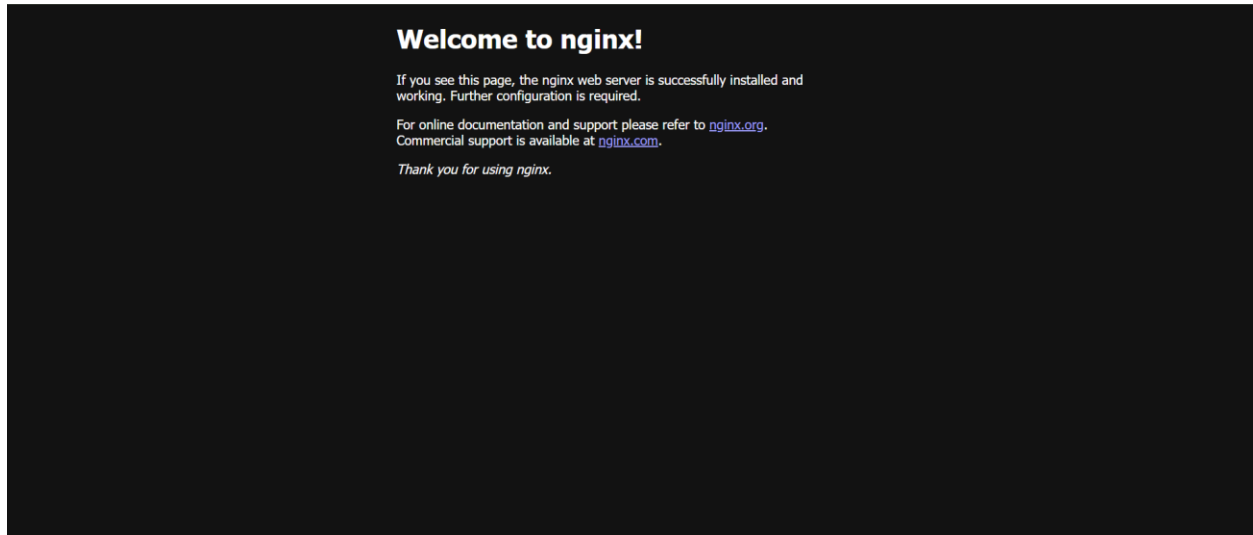


Figure 6: Verify the NGINX server

- Alternatively, run:

docker ps

```
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a20cd0e76442	a830707172e8	"/docker-entrypoint..."	3 minutes ago	Up 3 minutes	0.0.0.0:8000->80/tcp	tutorial
2c5826607716	moby/buildkit:buildx-stable-1	"buildkitd --allow-i..."	4 weeks ago	Up 6 minutes		buildx_buildkit_quirky_hopper0
4d9f2bc04d80	moby/buildkit:buildx-stable-1	"buildkitd --allow-i..."	4 weeks ago	Up 6 minutes		buildx_buildkit_mybuilder0

Figure 7: Display the created image with Docker

Step 7: Destroy the Container

- To stop and remove the NGINX container:

terraform destroy

```
Command Prompt - terraform x + v
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>terraform destroy
docker_image.nginx: Refreshing state... [id=sha256:a830707172e8069c09cf6c67a04e23e5a1a332d70a90a54999b76273a928b9cenginx]
docker_container.nginx: Refreshing state... [id=a20cd0e76442288b6ebb01acc01fdbcb874e42d02ad21873095304d2453618a8c]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# docker_container.nginx will be destroyed
- resource "docker_container" "nginx" {
  - attach                = false -> null
  - command               = [
    - "nginx",
    - "-g",
    - "daemon off;",
  ] -> null
  - container_read_refresh_timeout_milliseconds = 15000 -> null
  - cpu_shares             = 0 -> null
  - dns                   = [] -> null
  - dns_opts              = [] -> null
  - dns_search             = [] -> null
  - entrypoint             = [
    - "/docker-entrypoint.sh",
  ] -> null
  - env                   = [] -> null
  - group_add             = [] -> null
  - hostname              = "a20cd0e76442288b6ebb01acc01fdbcb874e42d02ad21873095304d2453618a8c" -> null
  - id                    = "a20cd0e76442288b6ebb01acc01fdbcb874e42d02ad21873095304d2453618a8c" -> null
  - image                 = "sha256:a830707172e8069c09cf6c67a04e23e5a1a332d70a90a54999b76273a928b9c" -> null
  - init                  = false -> null
  - ipc_mode              = "private" -> null
  - log_driver             = "json-file" -> null
  - log_opts              = {} -> null
  - logs                  = false -> null
  - max_retry_count       = 0 -> null
  - memory                = 0 -> null
  - memory_swap           = 0 -> null
  - must_run              = true -> null
  - name                  = "tutorial" -> null
}
```

Figure 8: Stop and remove the container

- Confirm by typing 'yes' when prompted.

```
Command Prompt x + v
- storage_opts           = {} -> null
- sysctls                = {} -> null
- tmpfs                  = {} -> null
- tty                    = false -> null
- wait                   = false -> null
- wait_timeout           = 60 -> null
# (7 unchanged attributes hidden)

- ports {
  - external = 8000 -> null
  - internal = 80 -> null
  - ip       = "0.0.0.0" -> null
  - protocol = "tcp" -> null
}

# docker_image.nginx will be destroyed
- resource "docker_image" "nginx" {
  - id          = "sha256:a830707172e8069c09cf6c67a04e23e5a1a332d70a90a54999b76273a928b9cenginx" -> null
  - image_id    = "sha256:a830707172e8069c09cf6c67a04e23e5a1a332d70a90a54999b76273a928b9c" -> null
  - keep_locally = false -> null
  - name        = "nginx" -> null
  - repo_digest = "nginx@sha256:c15da6c91de8d2f436196f3a768483ad32c258ed4e1beb3d367a27ed67253e66" -> null
}

Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

docker_container.nginx: Destroying... [id=a20cd0e76442288b6ebb01acc01fdbcb874e42d02ad21873095304d2453618a8c]
docker_image.nginx: Destruction complete after 1s
docker_image.nginx: Destroying... [id=sha256:a830707172e8069c09cf6c67a04e23e5a1a332d70a90a54999b76273a928b9cenginx]
docker_image.nginx: Destruction complete after 0s

Destroy complete! Resources: 2 destroyed.

C:\Users\Crytek Work Xpace\learn-terraform-docker-container>
```

Figure 9: Stop and remove the container

- Check with docker:

docker ps

```
C:\Users\Crytek Work Xpace\learn-terraform-docker-container>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2c5826687716	moby/buildkit:buildx-stable-1	"buildkitd --allow-i_"	4 weeks ago	Up 8 minutes		buildx_buildkit_quirky_hopper0
4d9f2bc04d80	moby/buildkit:buildx-stable-1	"buildkitd --allow-i_"	4 weeks ago	Up 8 minutes		buildx_buildkit_mybuilder0

Figure 10: Check with Docker

Conclusion

By following this tutorial, I successfully provisioned and managed a containerized NGINX web server using Terraform and Docker on my Windows machine. This hands-on practice helped me understand how powerful and straightforward Infrastructure as Code (IaC) can be, using code instead of manual steps to create, deploy, and tear down infrastructure.

This is just the beginning. From here, I can continue improving my Terraform skills by handling more complex setups, trying out different Docker images, or even working with cloud platforms like AWS or Azure. Writing infrastructure as code makes everything more consistent, easier to manage, and repeatable.

I'll keep exploring, building, and learning. DevOps and automation have a lot to offer, and this was a great first step.